

CS162
Operating Systems and
Systems Programming
Lecture 11

Scheduling 3:
Classic Policies

October 13th, 2025

Yifan Qiao

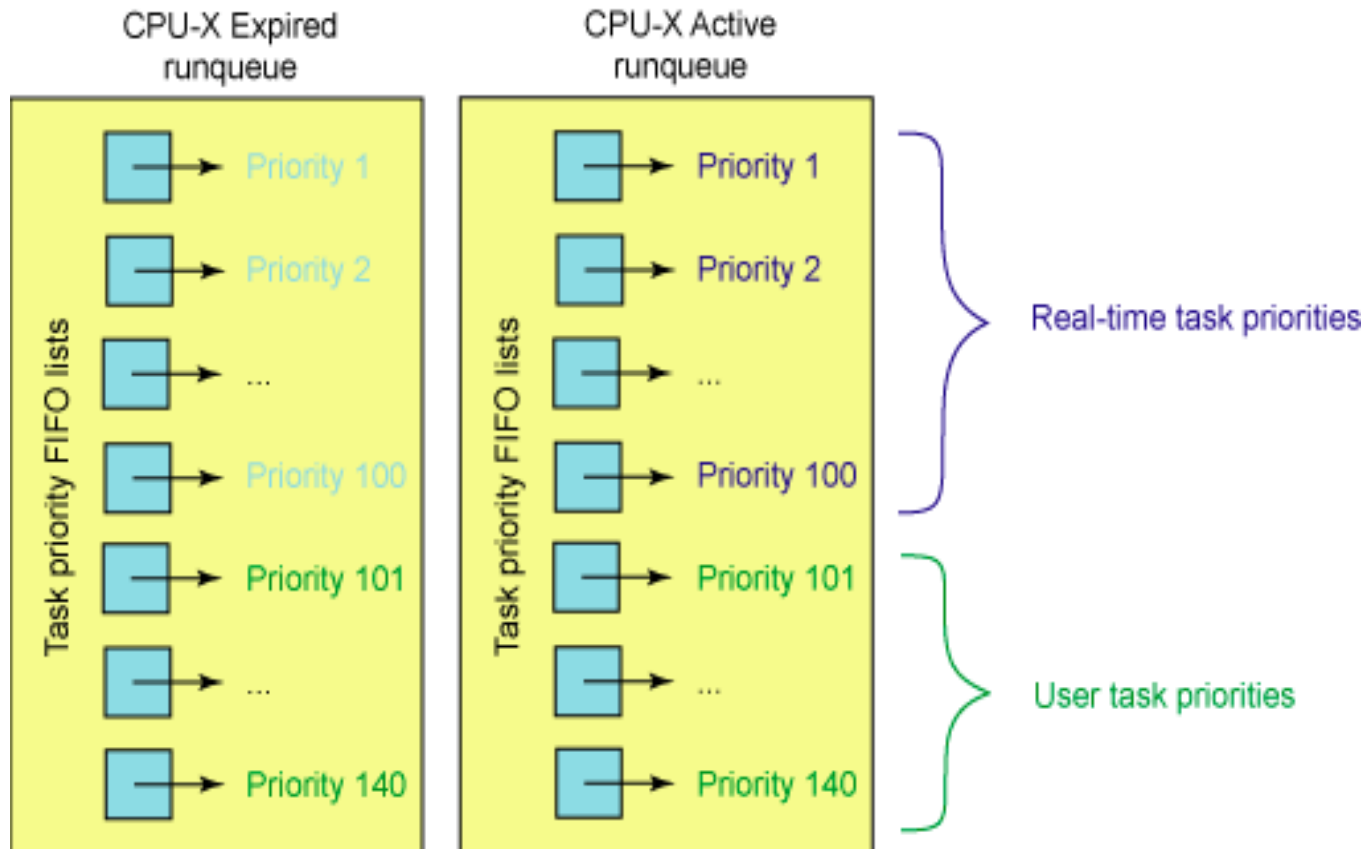
<http://cs162.eecs.Berkeley.edu>

Case Study: Linux O(1) Scheduler



- Priority-based scheduler: 140 priorities
 - 40 for “user tasks” (set by “nice”), 100 for “Realtime/Kernel”
 - Lower priority value $\Rightarrow$ higher priority (for nice values)
 - Highest priority value $\Rightarrow$ Lower priority (for realtime values)
 - All algorithms $O(1)$
 - » Timeslices/priorities/interactivity credits all computed when job finishes time slice
 - » 140-bit bit mask indicates presence or absence of job at given priority level
- Two separate priority queues: “active” and “expired”
 - All tasks in the active queue use up their timeslices and get placed on the expired queue, after which queues swapped
- Timeslice depends on priority – linearly mapped onto timeslice range
 - Like a multi-level queue (one queue per priority) with different timeslice at each level
 - Execution split into “Timeslice Granularity” chunks – round robin through priority

Linux O(1) Scheduler



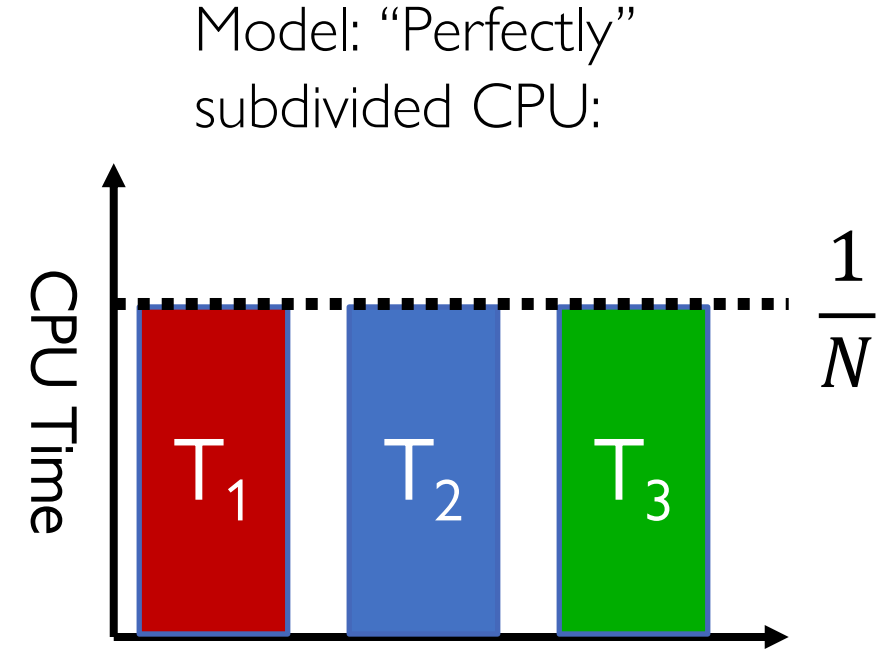
- Lots of ad-hoc heuristics
 - Try to boost priority of I/O-bound tasks
 - Try to boost priority of starved tasks

O(1) Scheduler Continued

- Heuristics
 - User-task priority adjusted ± 5 based on heuristics
 - » $p \rightarrow \text{sleep_avg} = \text{sleep_time} - \text{run_time}$
 - » Higher sleep_avg $\Rightarrow$ more I/O bound the task, more reward (and vice versa)
 - Interactive Credit
 - » Earned when a task sleeps for a “long” time
 - » Spend when a task runs for a “long” time
 - » IC is used to provide hysteresis to avoid changing interactivity for temporary changes in behavior
 - However, “interactive tasks” get special dispensation
 - » To try to maintain interactivity
 - » Placed back into active queue, unless some other task has been starved for too long...
- Real-Time Tasks
 - Always preempt non-RT tasks
 - No dynamic adjustment of priorities
 - Scheduling schemes:
 - » SCHED_FIFO: preempts other tasks, no timeslice limit
 - » SCHED_RR: preempts normal tasks, RR scheduling amongst tasks of same priority

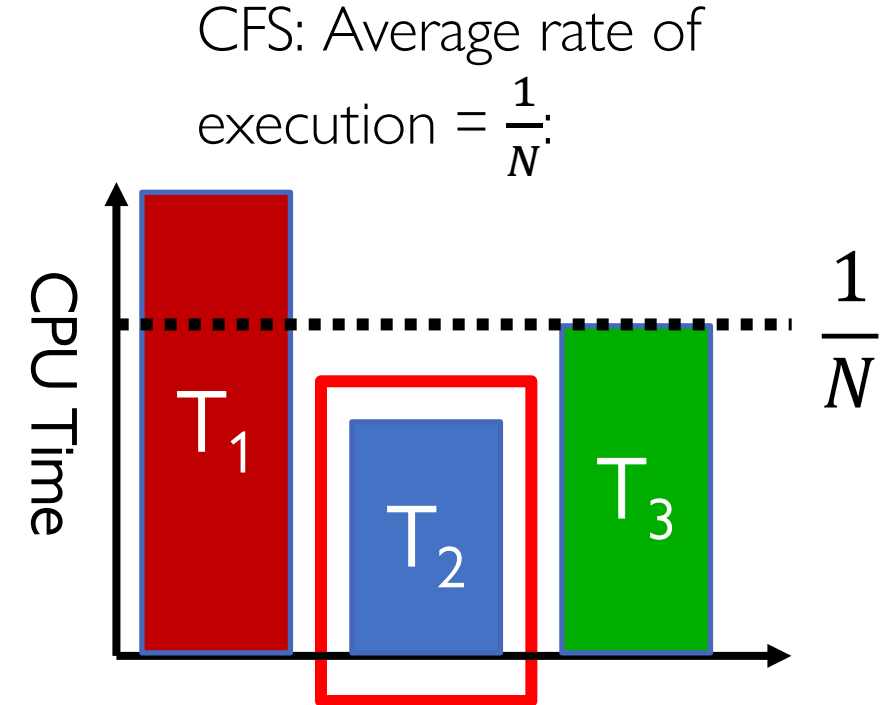
Linux Completely Fair Scheduler (CFS)

- Goal: Each process gets an equal share of CPU
 - N threads “simultaneously” execute on $\frac{1}{N}$ of CPU
 - The *model* is somewhat like simultaneous multithreading
 - each thread gets $\frac{1}{N}$ of the cycles
- In general, can't do this with real hardware
 - OS needs to give out full CPU in time slices
 - Thus, we must use something to keep the threads roughly in sync with one another



Linux Completely Fair Scheduler (CFS)

- Basic Idea: track CPU time per thread and schedule threads to match up average rate of execution
- Scheduling Decision:
 - “Repair” illusion of complete fairness
 - Choose thread with minimum CPU time
 - Closely related to Fair Queueing
- Use a heap-like scheduling queue for this...
 - $O(\log N)$ to add/remove threads, where N is number of threads
- Sleeping threads don’t advance their CPU time, so they get a boost when they wake up again...
 - Get interactivity automatically!



Linux CFS: Responsiveness/Starvation Freedom

- In addition to fairness, we want **low response time** and starvation freedom
 - Make sure that everyone gets to run at least a bit!
- Constraint 1: *Target Latency*
 - Period of time over which every process gets service
 - $\text{Quanta} = \text{Target_Latency} / n$
- Target Latency: 20 ms, 4 Processes
 - Each process gets 5ms time slice
- Target Latency: 20 ms, 200 Processes
 - Each process gets 0.1ms time slice (!!!)
 - Recall Round-Robin: large context switching overhead if slice gets too small

Linux CFS: Throughput

- Goal: Throughput
 - Avoid excessive overhead
- Constraint 2: Minimum Granularity
 - Minimum length of any time slice
- Target Latency 20 ms, Minimum Granularity 1 ms, 200 processes
 - Each process gets 1 ms time slice

Aside: Priority in Unix – Being Nice

- The industrial operating systems of the 60s and 70's provided priority to enforced desired usage policies.
 - When it was being developed at Berkeley, instead it provided ways to “be nice”.
- **nice** values range from -20 to 19
 - Negative values are “not nice”
 - If you wanted to let your friends get more time, you would nice up your job
- Scheduler puts higher nice-value tasks (lower priority) to sleep more ...
 - In $O(1)$ scheduler, this translated fairly directly to priority (and time slice)
- How does this idea translate to CFS?
 - Change the rate of CPU cycles given to threads to change relative priority

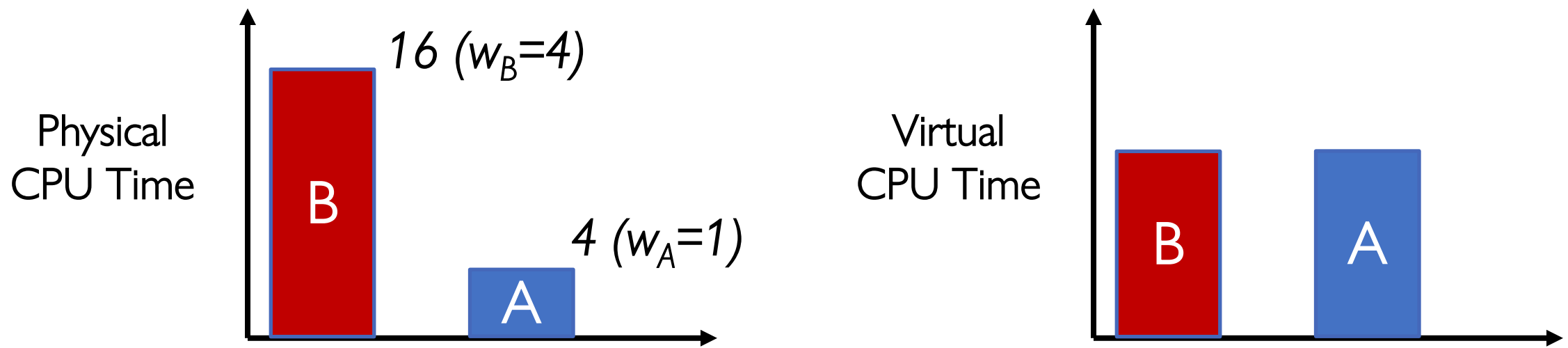
Linux CFS: Proportional Shares

- What if we want to give more CPU to some and less to others in CFS (proportional share) ?
 - Allow different threads to have different *rates* of execution (cycles/time)
- Use weights! Key Idea: Assign a weight w_i to each process i to compute the switching quanta Q_i
 - Basic equal share: $Q_i = \text{Target Latency} \cdot \frac{1}{N}$
 - Weighted Share: $Q_i = \left(w_i / \sum_p w_p \right) \cdot \text{Target Latency}$
- Reuse **nice** value to reflect share, rather than priority,
 - Remember that lower nice value $\Rightarrow$ higher priority
 - CFS uses nice values to scale weights exponentially: $\text{Weight} = 1024 / (1.25)^{\text{nice}}$
 - » Two CPU tasks separated by nice value of 5 $\Rightarrow$
Task with lower nice value has 3 times the weight, since $(1.25)^5 \approx 3$
- So, we use “Virtual Runtime” instead of CPU time

Example: Linux CFS: Proportional Shares

- Target Latency = 20ms
- Minimum Granularity = 1ms
- Example: Two CPU-Bound Threads
 - Thread A has weight 1
 - Thread B has weight 4
- Time slice for A? 4 ms
- Time slice for B? 16 ms

Linux CFS: Proportional Shares



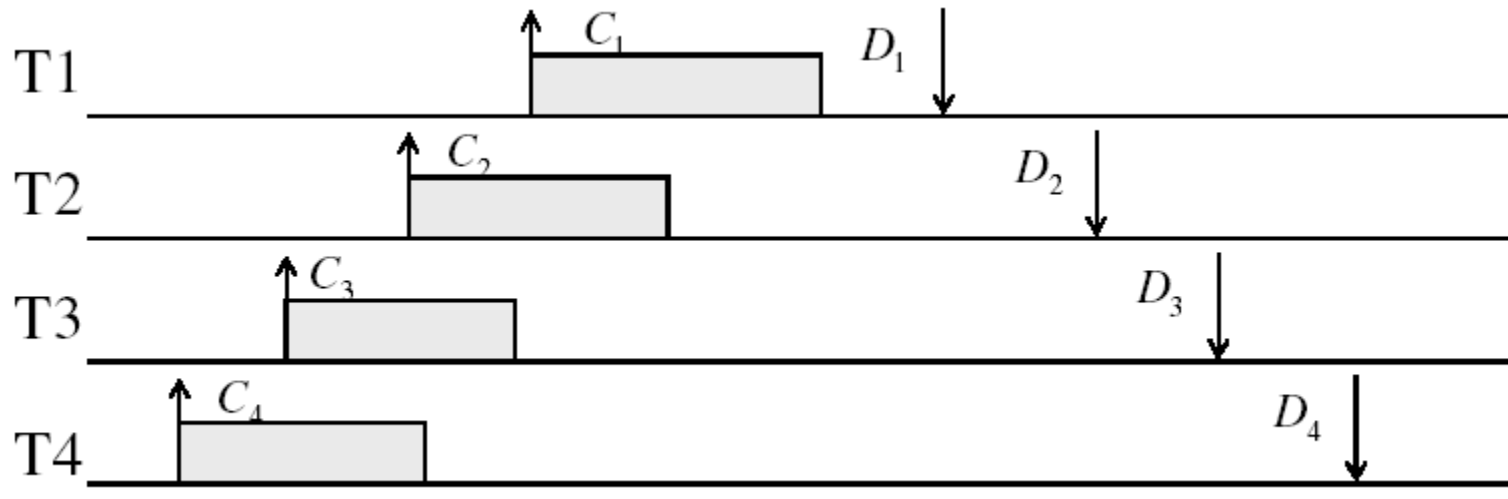
- Track a thread's *virtual* runtime rather than its true physical runtime
 - Higher weight: Virtual runtime increases more slowly
 - Lower weight: Virtual runtime increases more quickly
- Scheduler's Decisions are based on Virtual CPU Time
- Use of Red-Black tree to hold all runnable processes as sorted on vruntime variable
 - $O(1)$ time to find next thread to run (top of heap!)
 - $O(\log N)$ time to perform insertions/deletions
 - » Cash the item at far left (item with earliest vruntime)
 - When ready to schedule, grab version with smallest vruntime (which will be item at the far left).

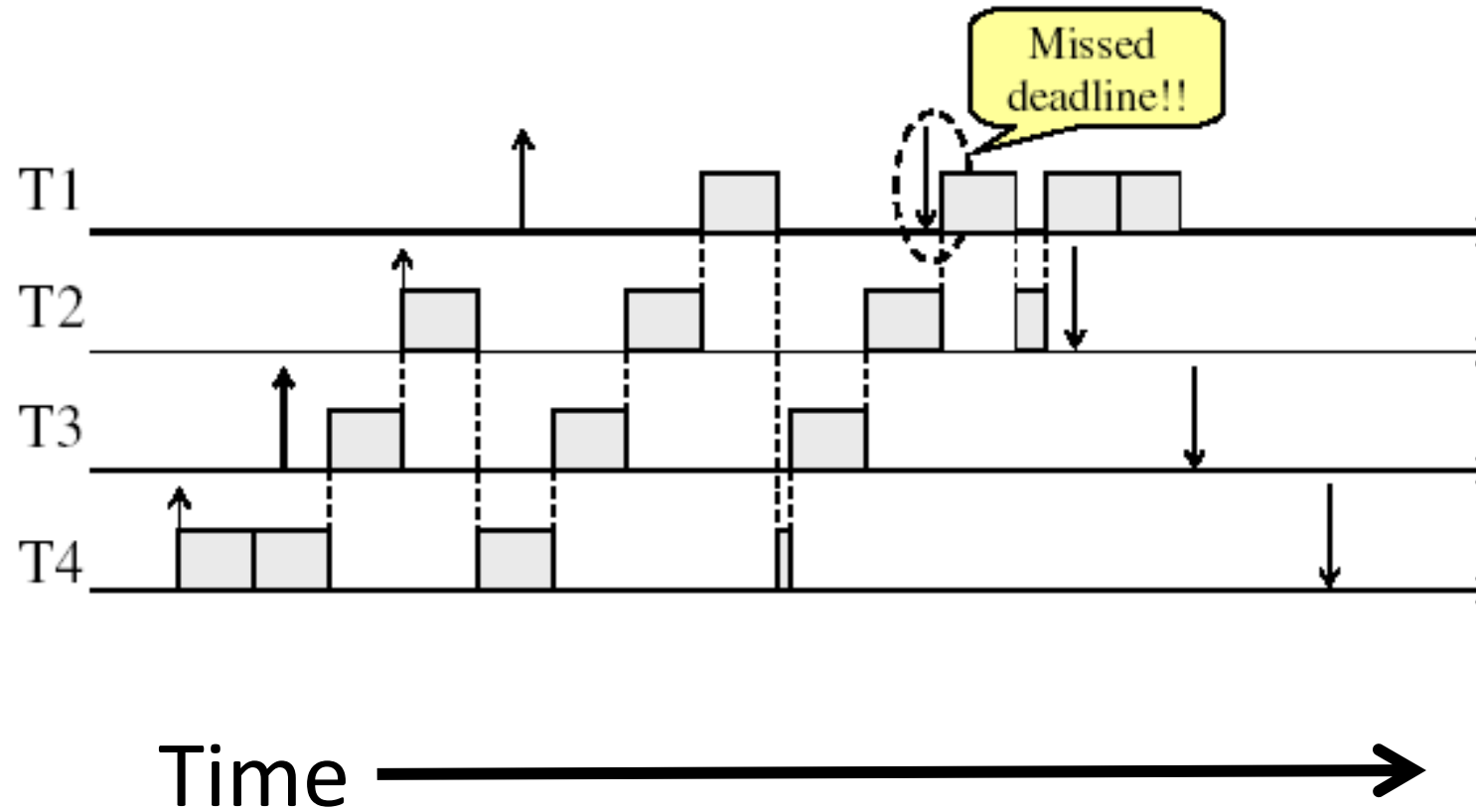
Real-Time Scheduling

- Goal: **Predictability** of Performance!
 - We need to predict with confidence worst case response times for systems!
 - In RTS, performance guarantees are:
 - » Task- and/or class centric and often ensured a priori
 - In conventional systems, performance is:
 - » System/throughput oriented with post-processing (...wait and see ...)
 - Real-time is about enforcing predictability, and does not equal fast computing!!!
- Hard real-time: for time-critical safety-oriented systems
 - Meet all deadlines (if at all possible)
 - Ideally: determine in advance if this is possible
 - **Earliest Deadline First (EDF), Least Laxity First (LLF), Rate-Monotonic Scheduling (RMS), Deadline Monotonic Scheduling (DM)**
- Soft real-time: for multimedia
 - Attempt to meet deadlines with high probability
 - **Constant Bandwidth Server (CBS)**

Example: Workload Characteristics

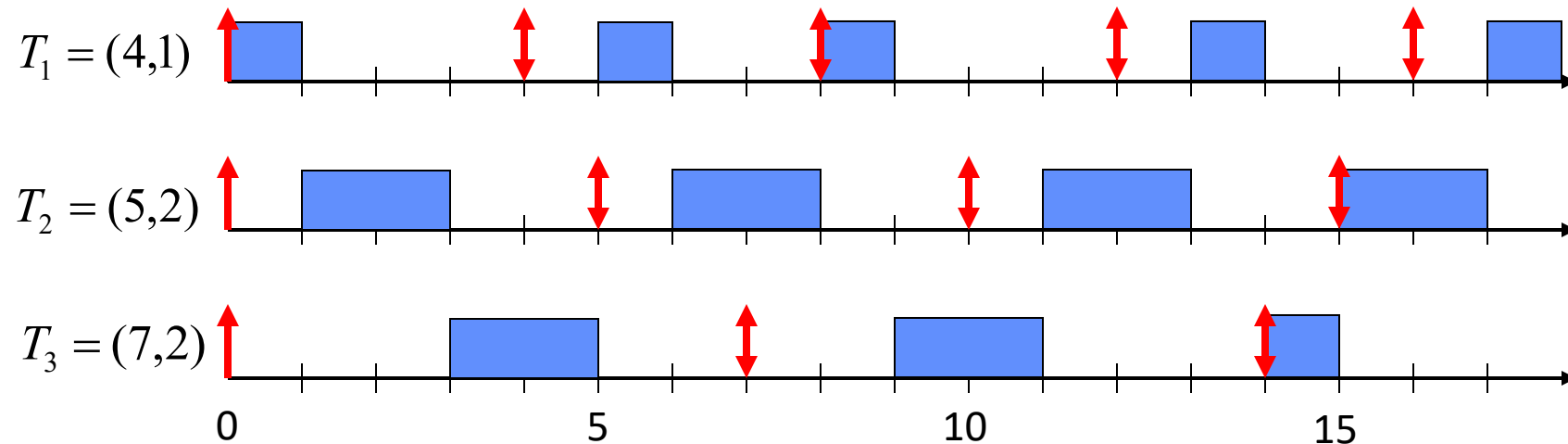
- Tasks are preemptable, independent with arbitrary arrival (=release) times
- Tasks have deadlines (D) and known computation times (C)
- Example Setup:





Earliest Deadline First (EDF)

- Tasks **periodic** with period P and computation C in each period: (P_i, C_i) for each task i
- Preemptive priority-based dynamic scheduling:
 - Each task is assigned a (current) priority based on how close the absolute deadline is (i.e. $D_i^{t+1} = D_i^t + P_i$ for each task!)
 - The scheduler always schedules the active task with the closest absolute deadline



EDF Feasibility Testing

- Even EDF won't work if you have too many tasks
- For n tasks with computation time C and deadline D , a feasible schedule exists if:

$$\sum_{i=1}^n \left(\frac{C_i}{D_i} \right) \leq 1$$

Multi-Core Scheduling

- Algorithmically, not a huge difference from single-core scheduling
- Implementation-wise, helpful to have *per-core* scheduling data structures
 - Cache coherence
- *Affinity scheduling*: once a thread is scheduled on a CPU, OS tries to reschedule it on the same CPU
 - Cache reuse, branch prediction
 - Example for $O(1)$ scheduler: one set of queues/core with background rebalancing

Spinlocks for multiprocessing

- Spinlock implementation:

```
int value = 0; // Free
Acquire() {
    while (test&set(&value)) {}; // spin while busy
}
Release() {
    value = 0; // atomic store
}
```

- Spinlock doesn't put the calling thread to sleep—it just busy waits
 - When might this be preferable?
 - » Waiting for limited number of threads at a barrier in a multiprocessing (multicore) program
 - » Wait time at barrier would be greatly increased if threads must be woken inside kernel
- Every **test&set()** atomic read & write
 - Makes value ping-pong around between core-local caches (using lots of memory!)

Spinlocks for multiprocessing (cont'd)

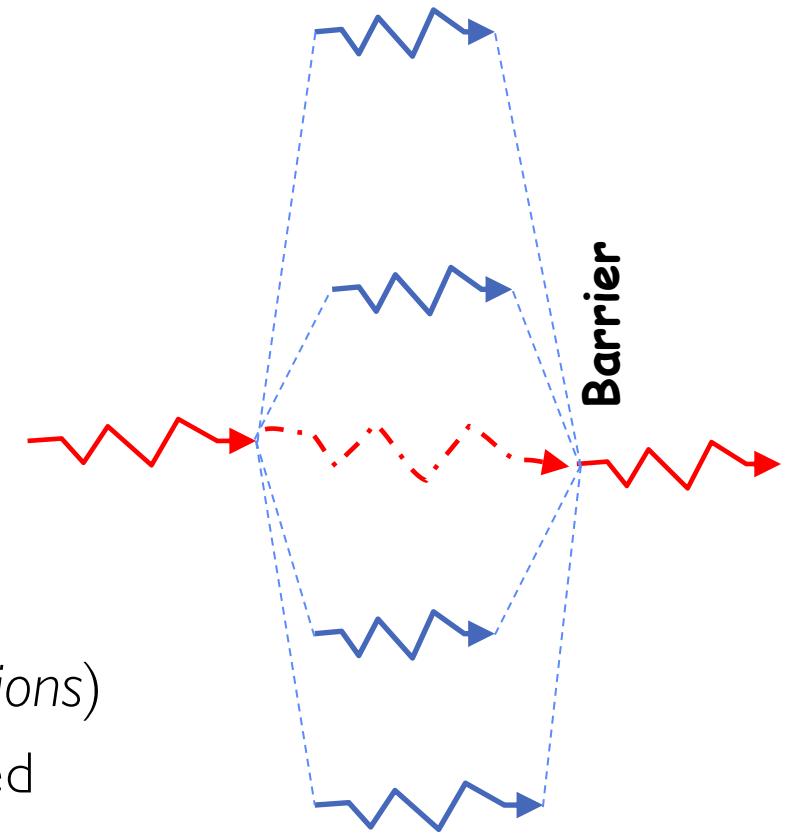
- Want **test&test&set()** !
 - First test just reads the lock; no memory contention!
 - Only if free do atomic **test&set()**

- The extra read eliminates the ping-ponging issues:

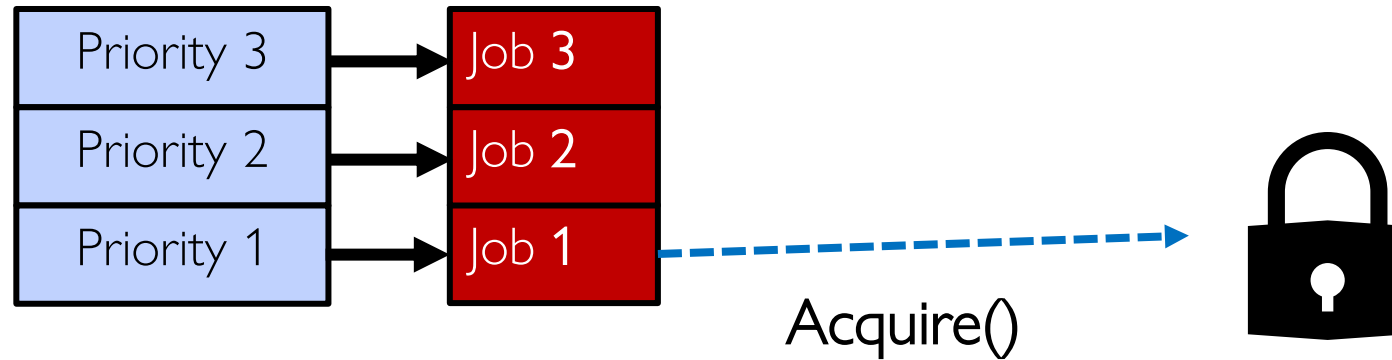
```
// Implementation of test&test&set():
Acquire() {
    do {
        while(value);           // wait until might be free
    } while (test&set(&value)); // exit if acquire lock
}
```

Gang Scheduling and Parallel Applications

- When multiple threads work together on a multi-core system, try to schedule them together
 - Makes spin-waiting more efficient (inefficient to spin-wait for a thread that's suspended)
 - Multiple phases of parallel and serial execution
- Additionally: OS informs a parallel program how many processors its threads are scheduled on (*Scheduler Activations*)
 - Application adapts to number of cores that it has scheduled
 - “Space sharing” with other parallel programs can be more efficient, because parallel speedup is often sublinear with the number of cores

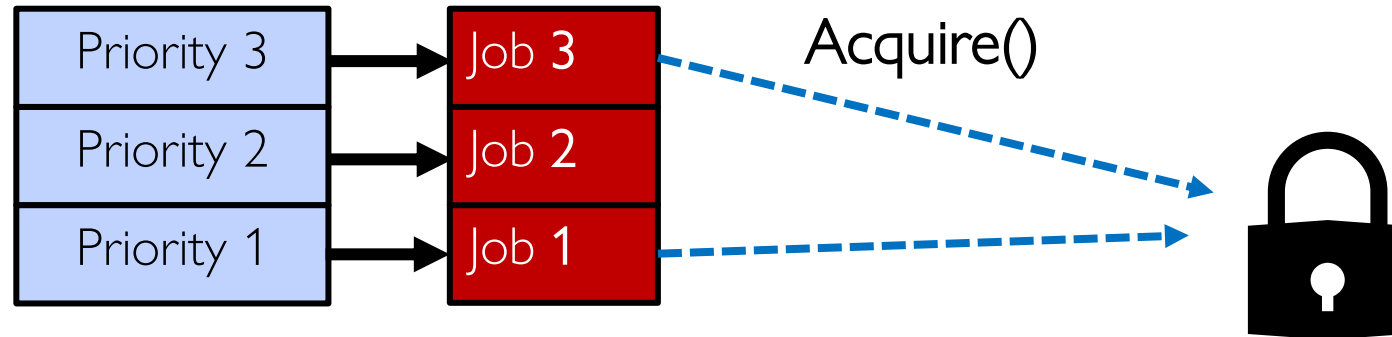


Priority Inversion



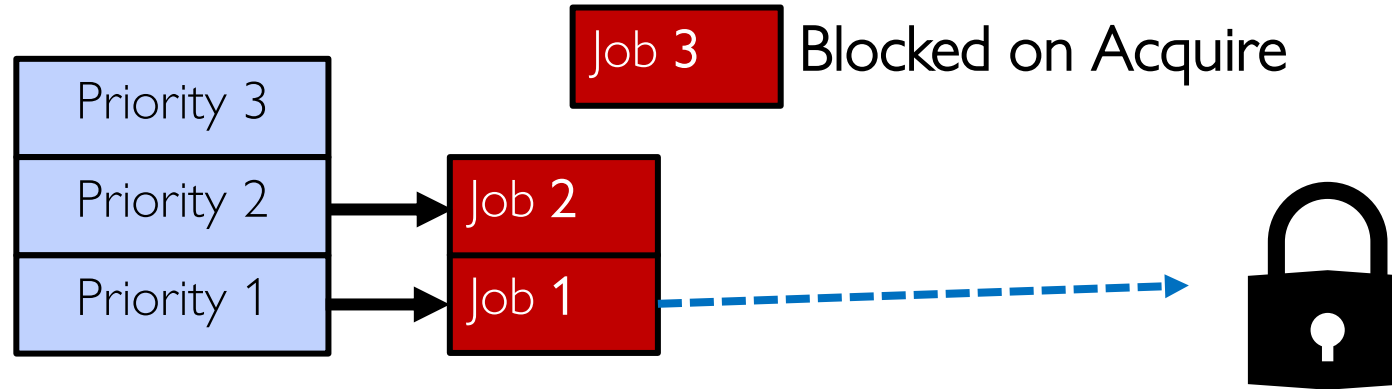
- At this point, which job does the scheduler choose?
- Job 3 (Highest priority)

Priority Inversion



- Job 3 attempts to acquire lock held by Job 1

Priority Inversion



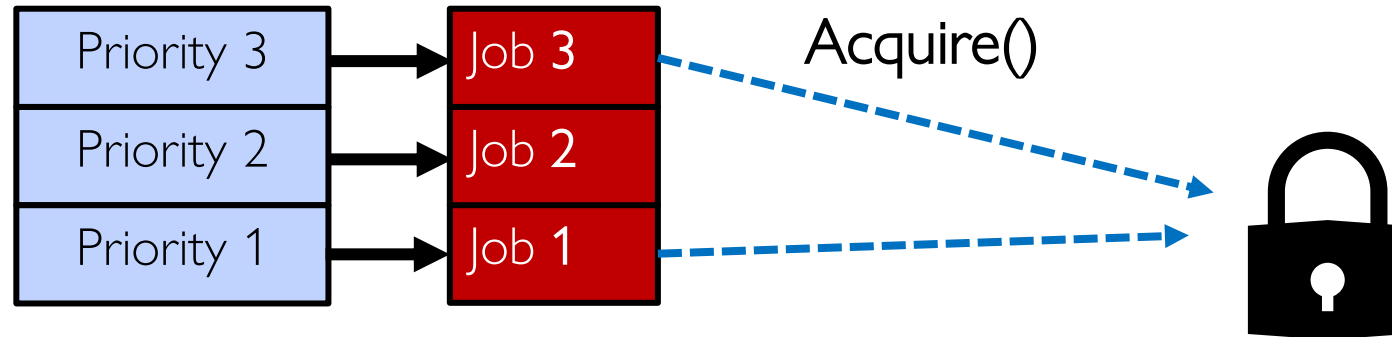
- At this point, which job does the scheduler choose?
- Job 2 (Medium Priority)
- Priority Inversion

Priority Inversion

- Where high priority task is blocked waiting on low priority task
- Low priority one *must* run for high priority to make progress
- Medium priority task can starve a high priority one
- When else might priority lead to starvation or “live lock”?

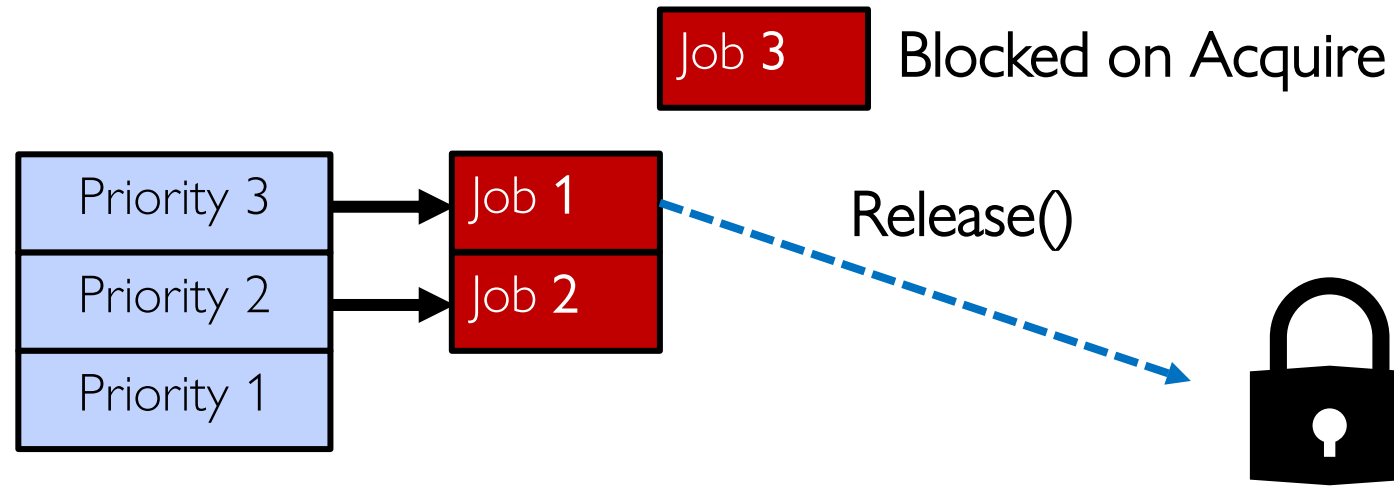


One Solution: Priority Donation/Inheritance



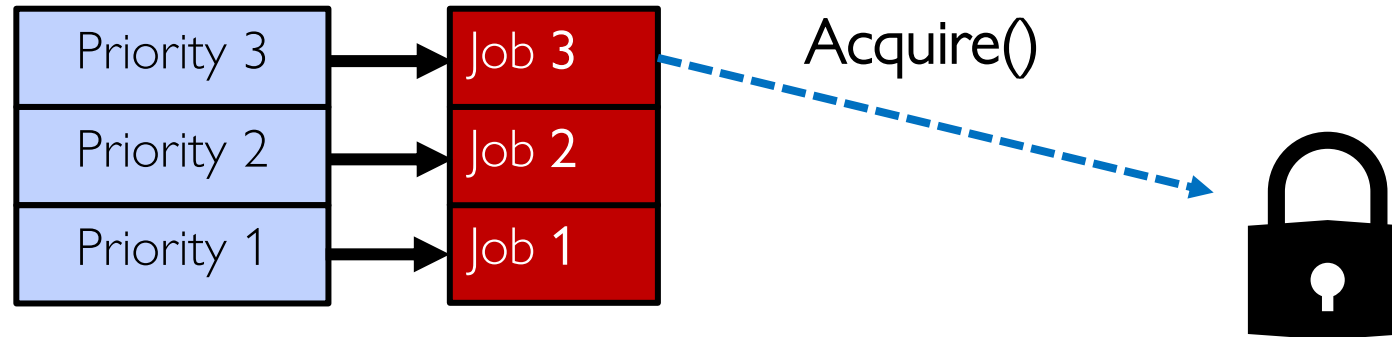
- Job 3 temporarily grants Job 1 its “high priority” to run on its behalf

One Solution: Priority Donation/Inheritance



- Job 3 temporarily grants Job 1 its “high priority” to run on its behalf

One Solution: Priority Donation/Inheritance



- Job 1 completes critical section and releases lock
- Job 3 acquires lock, runs again
- How does the scheduler know?



Case Study: Martian Pathfinder Rover

- July 4, 1997 – Pathfinder lands on Mars
 - First US Mars landing since Vikings in 1976; first rover
 - Novel delivery mechanism: inside air-filled balloons bounced to stop on the surface from orbit!



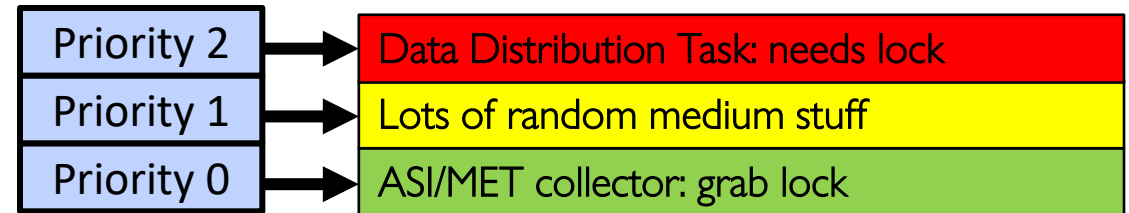
- And then...a few days into mission...:

- Multiple system resets occur to realtime OS (VxWorks)
- System would reboot randomly, losing valuable time and progress

<https://youtu.be/3E9BcdFYENg?t=122>

- Problem? Priority Inversion!

- Low priority task grabs mutex trying to communicate with high priority task:

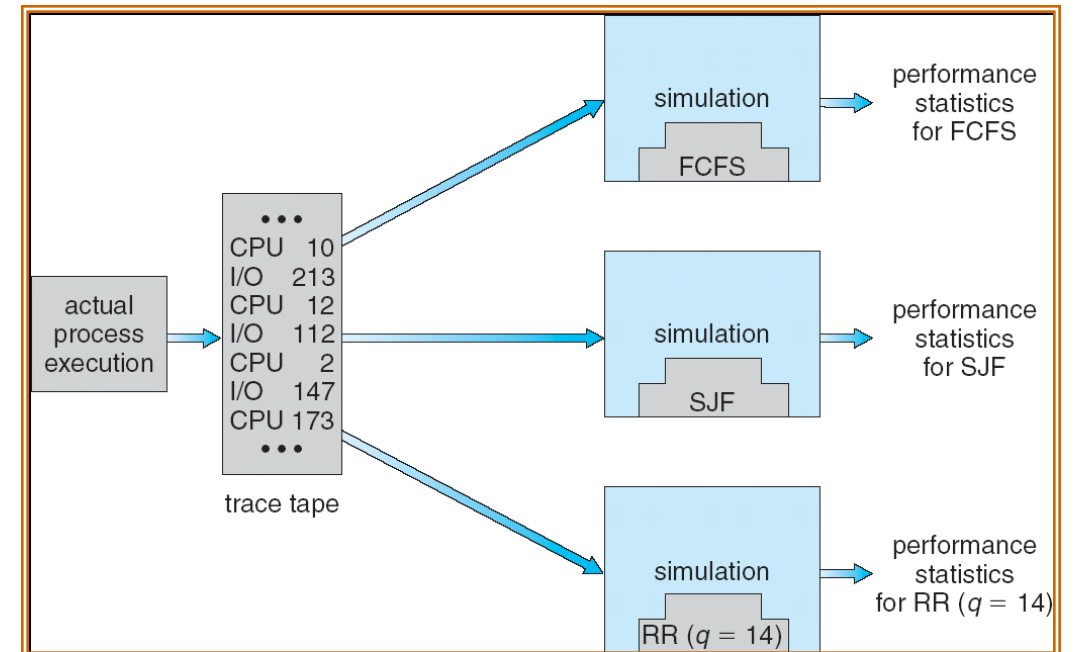


- Realtime watchdog detected lack of forward progress and invoked reset to safe state
 - » High-priority data distribution task was supposed to complete with regular deadline

- Solution: Turn priority donation back on and upload fixes!
- Original developers turned off priority donation (also called priority inheritance)
 - Worried about performance costs of donating priority!

How to Evaluate a Scheduling algorithm?

- Deterministic modeling
 - takes a predetermined workload and compute the performance of each algorithm for that workload
- Queueing models
 - Mathematical approach for handling stochastic workloads
- Implementation/Simulation:
 - Build system which allows actual algorithms to be run against actual data
 - Most flexible/general

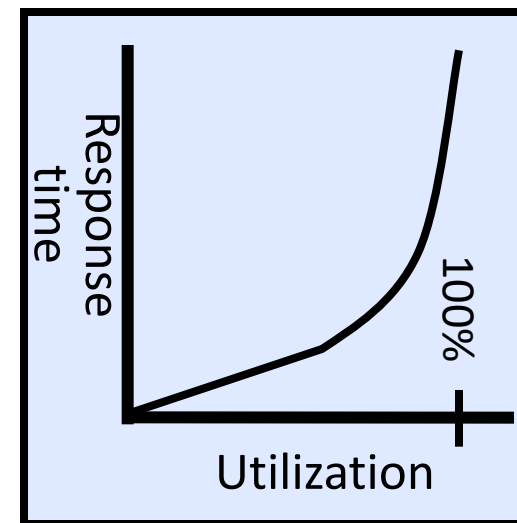


Choosing the Right Scheduler

I Care About:	Then Choose:
CPU Throughput	FCFS
Avg. Response Time	SRTF Approximation
I/O Throughput	SRTF Approximation
Fairness (CPU Time)	Round Robin, Fair Sharing, Lottery, EEVDF, ...
Meeting Deadlines	EDF
Favoring Important Tasks	Priority

A Final Word On Scheduling

- When do the details of the scheduling policy and fairness really matter?
 - When there aren't enough resources to go around
- When should you simply buy a faster computer?
 - (Or network link, or expanded highway, or ...)
 - One approach: Buy it when it will pay for itself in improved response time
 - » Perhaps you're paying for worse response time in reduced productivity, customer angst, etc...
 - » Might think that you should buy a faster X when X is utilized 100%, but usually, response time goes to infinity as utilization $\Rightarrow$ 100%
- An interesting implication of this curve:
 - Most scheduling algorithms work fine in the “linear” portion of the load curve, fail otherwise
 - Argues for buying a faster X when hit “knee” of curve



Conclusion

- Realtime Schedulers such as EDF
 - Guaranteed behavior by meeting deadlines
 - Realtime tasks defined by tuple of compute time and period
 - Schedulability test (admission control): is it possible to meet deadlines with proposed set of processes?
- Fair Sharing Scheduling
 - Give each job a share of the CPU according to its priority
 - Low-priority jobs get to run less often
 - But all jobs can at least make progress (no starvation)
- Priority Inversion
 - A higher-priority task is prevented from running by a lower-priority task
 - Often caused by locks and through the intervention of a middle-priority task